VOLUME III · THE GAPS · ALGORITHMS, ARRAYS, ERRORS AND THE PROGRAM LIBRARY

CMS 708 — What the Course Document Leaves Out

The four modules cover control structures and functions well, but a structured-programming paper reaches wider: pseudocode and flowcharts, the structured programming theorem, arrays, the three error types, storage classes, and the dozen standard programs every such exam draws from. All of it is here, worked.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Monday 10 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer

HOW TO READ THIS VOLUME

Sections **§1–§3** are **theory the course document alludes to but never states** — the three constructs, algorithm design, the error types. They are cheap marks and are asked in almost every structured-programming paper. Sections **§4–§6** are **C++ features the exercises silently require** — `static` is set as exercise 21 but never taught, and arrays underpin most "write a program" questions. Section **§7** is a **library of standard programs**. Learn §1–§3 first; they are the fastest marks in the volume.

1 The structured programming theorem

The course document says structured programming uses "clear control structures" but never states the underlying result. It is worth quoting because it explains **why** `goto` can simply be banned:

> **The structured programming theorem** (Böhm and Jacopini, 1966): **any computable function can be written using only three control structures — sequence, selection and iteration**. No `goto` is ever necessary.

CONSTRUCT	MEANING	IN C++
Sequence	Statements executed **one after another** in the order written	Ordinary statements, one per line
Selection	A **choice** between two or more alternative paths	`if` · `if–else` · `switch`
Iteration	**Repetition** of a block while a condition holds	`for` · `while` · `do–while`

Each construct has **one entry point and one exit point** — that single-entry / single-exit property is what makes structured programs traceable, and what `goto` destroys.

COHESION AND COUPLING — THE MEASURE OF GOOD MODULES

Modularity is examinable, and these two words are how its quality is judged:

TERM	DEFINITION	YOU WANT
Cohesion	How **strongly the tasks inside one module belong together**. A function that does exactly one job is highly cohesive	**High**
Coupling	How **dependent modules are on one another**. Functions that communicate only through parameters and return values are loosely coupled	**Low**

> **The rule to quote:** good modular design aims for **high cohesion and low coupling** — each module does one thing well and depends on the others as little as possible. Global variables are the classic cause of **tight coupling**, which is one more reason to prefer parameters.

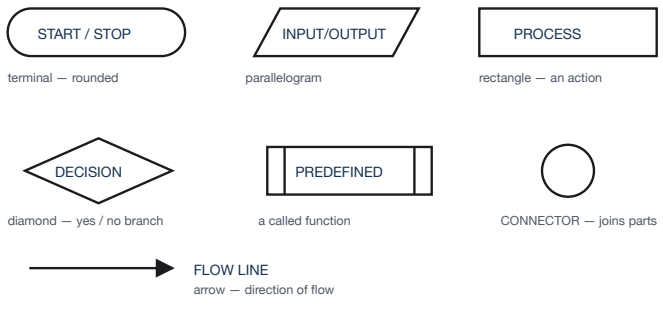
ADVANTAGES OF FUNCTIONS — A LIKELY LIST QUESTION

1. **Reusability** — write once, call many times
2. **Reduces code length** and eliminates repetition
3. **Easier debugging** — faults are isolated to one function
4. **Divides labour** — different programmers take different functions
5. **Improves readability** — `main()` reads like a summary of the program
6. **Easier maintenance** — a change is made in one place only

2 Algorithm design — pseudocode and flowcharts

Before code comes the **algorithm**: a **finite, ordered sequence of unambiguous steps that solves a problem**. Papers on this course routinely ask you to **write an algorithm or draw a flowchart** before, or instead of, the program. Its properties: **finiteness** (it terminates), **definiteness** (each step is unambiguous), **input**, **output** and **effectiveness**.

THE FLOWCHART SYMBOLS YOU MUST BE ABLE TO DRAW



Six symbols cover every flowchart this paper can ask for.

THE SAME PROBLEM THREE WAYS — LARGEST OF TWO NUMBERS

```
ALGORITHM (numbered steps)
Step 1: Start
Step 2: Read A and B
Step 3: If A > B then print A
Step 4: Else print B
Step 5: Stop
```

PSEUDOCODE CONVENTIONS

```
BEGIN
    READ  number
    IF number MOD 2 = 0 THEN
        PRINT "Even"
    ELSE
        PRINT "Odd"
    ENDIF
END
```

Use **BEGIN/END**, **READ** and **PRINT**, **IF…THEN…ELSE…ENDIF**, **WHILE… ENDWHILE**, **FOR…ENDFOR**, and **indent every nested block**. Pseudocode is language-independent: **no semicolons, no** `cout`.

PSEUDOCODE

```
BEGIN
    READ A, B
    IF A > B THEN
        PRINT A
    ELSE
        PRINT B
    ENDIF
END
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin  >> a >> b;

    if (a > b) cout << a << " is larger" << endl;
    else       cout << b << " is larger" << endl;
    return 0;
}
```

> **If a question says "write an algorithm", numbered steps or pseudocode are both acceptable — but do not write C++.** Conversely if it says "write a program", pseudocode scores nothing. Read which one is asked.

The program development life cycle

#	STAGE	WHAT HAPPENS
1	**Problem definition**	State exactly what the program must do — the inputs, the outputs and the constraints
2	**Problem analysis**	Identify the data needed, the processing required and the results expected
3	**Algorithm design**	Work out the logic as an algorithm, pseudocode or flowchart — **before** any code is written
4	**Coding**	Translate the algorithm into a programming language such as C++
5	**Testing and debugging**	Run with test data, find and remove errors
6	**Documentation**	Comments, user guides and technical notes so others can maintain it
7	**Maintenance**	Correct faults and adapt the program as requirements change over its life

3 Errors, testing and debugging

ERROR TYPE	WHAT IT IS	EXAMPLE
Syntax error	Breaks the **rules of the language**. Caught by the **compiler**, so the program will not run at all	Missing semicolon · `cin << x;` · undeclared variable
Logic error	The program **compiles and runs** but produces the **wrong result**. Nothing warns you	`if (x = 5)` · using + where * was meant · summing all integers and calling it a sum of primes
Runtime error	Compiles, but **fails during execution**	Division by zero · infinite recursion · reading past the end of an array

> **The sentence examiners want: logic errors are the most dangerous**, because the compiler cannot detect them and the program appears to work — only testing against known correct results exposes them.

DEBUGGING TECHNIQUES

- **Desk checking / dry running** — trace the code by hand with sample values, in a table of variables
- **Print statements** — insert temporary `cout` lines to reveal intermediate values
- **Breakpoints and single-stepping** in a debugger
- **Divide and conquer** — comment out parts until the fault region is isolated
- **Rubber-duck explanation** — read the code aloud line by line; the mismatch usually announces itself

TEST DATA — THREE KINDS, AND YOU MUST NAME ALL THREE

KIND	PURPOSE	FOR "AGE ≥ 18"
Normal / valid	Typical values the program should accept	25, 40
Boundary / extreme	Values **at the edge** of acceptability, where off-by-one errors live	**17, 18, 19**
Invalid / erroneous	Values that must be **rejected gracefully**	−5, "abc", 0

A DRY-RUN TABLE — THE FORMAT TO USE

```
for (int i = 1; i <= 4; i++) { sum += i; }

 i  | sum before | sum after | i <= 4 ?
----+------------+-----------+----------
 1  |     0      |     1     |  true
 2  |     1      |     3     |  true
 3  |     3      |     6     |  true
 4  |     6      |    10     |  true
 5  |    10      |    10     |  FALSE → exit

Final: sum = 10, i = 5
```

Draw this table whenever a question says "trace", "dry run" or "what is the output". One column per variable, one row per iteration, and a final line stating the values on exit. It is worth marks even if the final number is wrong.

4 Function features the exercises assume

FUNCTION PROTOTYPES — DECLARING BEFORE DEFINING

```
#include <iostream>
using namespace std;

int add(int a, int b);        // PROTOTYPE — ends with ;

int main() {
    cout << add(3, 4) << endl;    // legal: compiler knows it
    return 0;
}

int add(int a, int b) {       // DEFINITION, after main()
    return a + b;
}
```

C++ reads a file top to bottom, so a function must be **known before it is called**. Either define it above `main()`, or declare a **prototype** above and define it below. The prototype gives the **return type, name and parameter types**, and **ends with a semicolon**.

DEFAULT ARGUMENTS

```
double power(double base, int exp = 2) {   // default
    double result = 1;
    for (int i = 0; i < exp; i++) result *= base;
    return result;
}

power(5);       // 25   — uses exp = 2
power(5, 3);    // 125  — supplied value wins
```

FUNCTION OVERLOADING

```
int    maxOf(int a, int b)       { return a > b ? a : b; }
double maxOf(double a, double b) { return a > b ? a : b; }

maxOf(3, 7);      // calls the int version
maxOf(3.5, 7.1);  // calls the double version
```

Overloading = several functions sharing **one name** but differing in the **number or types of their parameters**. The compiler picks by matching the arguments. Note the **conditional operator** `a > b ? a : b` — a compact `if-else` that returns a value.

THE THIRD WAY TO PASS A PARAMETER — BY POINTER

The notes give **by value** and **by reference**. C++ has a third method that older exam questions still ask for:

```
void incrementByPointer(int *p) {   // takes an ADDRESS
    (*p)++;                         // * dereferences it
}

int main() {
    int num = 10;
    incrementByPointer(&num);       // & takes the address
    cout << num << endl;            // 11 — original changed
    return 0;
}
```

METHOD	SYNTAX	ORIGINAL CHANGED?
By value	`f(int x)` · `f(a)`	No
By reference	`f(int &x)` · `f(a)`	**Yes**
By pointer	`f(int *x)` · `f(&a)`	**Yes**

Remember the two symbols: `&a` means "the **address of** a"; `*p` means "the **value at** the address p".

STORAGE CLASSES — WHERE `STATIC` COMES FROM

CLASS	SCOPE	LIFETIME
`auto`	Local to its block (the default for locals)	While the block runs
`static`	**Local** — visible only inside the function	**Whole program** — keeps its value between calls
`extern`	Global, and visible in **other files** too	Whole program
`register`	Local; a hint to keep it in a CPU register	While the block runs

This is the answer to exercise 21, which the course document sets but never teaches. A **static local variable** has the **scope of a local and the lifetime of a global** — which is exactly what a call-counter needs.

Recursion vs iteration — a standard comparison question

BASIS	RECURSION	ITERATION
Definition	A function **calls itself** on a smaller sub-problem	A **loop** repeats a block
Termination	A **base case**	A condition that eventually fails
Memory	**Higher** — every call adds a stack frame	**Lower** — one set of variables
Speed	Slower — call overhead	Faster
Code length	**Shorter and closer to the mathematical definition**	Longer but more explicit
Failure mode	**Stack overflow** if the base case is missing	**Infinite loop** if the condition never fails
Best for	Trees, factorial, Fibonacci, divide-and-conquer	Simple counted repetition

```
FACTORIAL, both ways

int factRec(int n) {
    if (n == 0) return 1;
    return n * factRec(n - 1);
}
```

```
int factIter(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++)
        result *= i;
    return result;
}
```

5 Arrays and strings

Arrays are absent from the course document but appear in almost every structured-programming paper, because they are what makes a loop worth writing. An **array** is a **collection of elements of the same data type stored in contiguous memory locations and accessed by an index**.

DECLARING, FILLING AND READING

```
int scores[5];                      // 5 ints, indices 0..4
int marks[5] = {70, 65, 80, 45, 90}; // declare + initialise

cout << marks[0];    // 70  — FIRST element is index 0
cout << marks[4];    // 90  — LAST is size − 1
```

Indices start at 0. An array of size *n* has indices **0 to n−1**; `marks[5]` above is **out of bounds** and is a runtime error waiting to happen. This off-by-one is the single most examined array trap.

READING VALUES INTO AN ARRAY

```
#include <iostream>
using namespace std;

int main() {
    int n = 5, marks[5];

    for (int i = 0; i < n; i++) {
        cout << "Enter mark " << i + 1 << ": ";
        cin  >> marks[i];
    }

    for (int i = 0; i < n; i++) {
        cout << "Mark " << i + 1 << " = " << marks[i] << endl;
    }
    return 0;
}
```

TWO-DIMENSIONAL ARRAYS

```
int table[3][4];              // 3 rows, 4 columns

for (int r = 0; r < 3; r++)
    for (int c = 0; c < 4; c++)
        table[r][c] = (r + 1) * (c + 1);

cout << table[2][3];          // row 2, column 3 → 12
```

A 2-D array always needs nested loops — outer for rows, inner for columns. This is where the nested-loop material of Module 3 pays off.

PASSING AN ARRAY TO A FUNCTION

```
#include <iostream>
using namespace std;

double average(int arr[], int size) {   // note: [] and a size
    int total = 0;
    for (int i = 0; i < size; i++) total += arr[i];
    return (double)total / size;        // cast! else truncates
}

int main() {
    int marks[5] = {70, 65, 80, 45, 90};
    cout << "Average: " << average(marks, 5) << endl;  // 70
    return 0;
}
```

An array is always passed by reference in effect — what is really passed is the address of its first element, so the function **can modify the caller's array**, and its size must be passed separately because that information is lost.

STRINGS — THE OPERATIONS WORTH KNOWING

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string first = "Ada", last = "Okon";

    string full = first + " " + last;   // concatenation
    cout << full            << endl;    // Ada Okon
    cout << full.length()   << endl;    // 8
    cout << full.substr(0,3)<< endl;    // Ada
    cout << full[0]         << endl;    // A

    string line;
    getline(cin, line);     // reads a WHOLE line incl. spaces
    return 0;
}
```

The `cin >> name` trap. `cin >>` stops at the first **space**, so "Ada Okon" reads as just "Ada". To read a full name use **`getline(cin, name);`**. Several exercises in Volume II would misbehave with a two-word answer for exactly this reason — worth one sentence if a question asks about input.

6 `break`, `continue`, and compiling a multi-file program

KEYWORD	EFFECT	EXAMPLE
`break`	**Exits the loop or `switch` immediately**	Leaving a login loop once the password is right
`continue`	**Skips the rest of this iteration** and jumps to the next one	Skipping negative numbers while summing
`return`	Exits the **whole function** at once	Returning early from a validation function

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) continue;   // skip only 5
    if (i == 8) break;      // stop entirely at 8
    cout << i << " ";
}
// prints: 1 2 3 4 6 7
```

WHAT HAPPENS WHEN YOU COMPILE

```
source.cpp
    │  1. PREPROCESSING   handles #include and #define
    ▼
expanded source
    │  2. COMPILATION     C++ → assembly, syntax checked here
    ▼
assembly
    │  3. ASSEMBLY        assembly → object code (.o)
    ▼
object file
    │  4. LINKING         joins your object files + libraries
    ▼
executable
```

Where errors surface: a missing semicolon is caught at **compilation**; a function you declared but never defined is caught at **linking**; division by zero survives all four stages and appears only at **run time**. That mapping is a neat one-mark answer.

Note that both are **disciplined single-exit jumps**, unlike `goto` — they leave only the construct they are in, so the flow stays traceable.

MODULARISATION ACROSS FILES

```
// maths.h – the interface (prototypes only)
double add(double a, double b);
double multiply(double a, double b);

// maths.cpp – the implementation
#include "maths.h"
double add(double a, double b)      { return a + b; }
double multiply(double a, double b) { return a * b; }

// main.cpp – the user
#include <iostream>
#include "maths.h"
using namespace std;
int main() { cout << add(2, 3); return 0; }
```

This is **modularisation taken to its conclusion**: the header declares **what** a module offers, the .cpp file holds **how** it does it, and users include only the header. **Angle brackets `< >` for system libraries, double quotes `" "` for your own files.**

7 The standard program library

Almost every "write a program that…" question on a structured-programming paper is one of these twelve, or a light disguise of one. Each is given in the shortest correct form; assume `#include <iostream>` and `using namespace std;` above every one.

1 · LARGEST OF THREE NUMBERS

```
int a, b, c;
cin >> a >> b >> c;

if (a >= b && a >= c)      cout << a;
else if (b >= a && b >= c) cout << b;
else                       cout << c;
```

2 · SUM AND AVERAGE OF N NUMBERS

```
int n, x, sum = 0;
cout << "How many numbers? ";  cin >> n;

for (int i = 0; i < n; i++) { cin >> x; sum += x; }

cout << "Sum: "     << sum << endl;
cout << "Average: " << (double)sum / n << endl;
```

3 · FACTORIAL, ITERATIVELY

```
int n, fact = 1;
cin >> n;
for (int i = 1; i <= n; i++) fact *= i;
cout << n << "! = " << fact << endl;
```

4 · PRIME CHECK

```
int n;  bool prime = true;
cin >> n;

if (n < 2) prime = false;
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) { prime = false; break; }
}
cout << (prime ? "Prime" : "Not prime") << endl;
```

5 · REVERSE A NUMBER

```
int n, reversed = 0;
cin >> n;

while (n > 0) {
    reversed = reversed * 10 + n % 10;  // take last digit
    n = n / 10;                         // drop last digit
}
cout << "Reversed: " << reversed << endl;
```

The two-line idiom — `n % 10` gives the last digit, `n / 10` removes it — also answers "sum of digits", "count digits" and "palindrome number".

6 · MULTIPLICATION TABLE FOR ONE NUMBER

```
int n;
cin >> n;
for (int i = 1; i <= 12; i++)
    cout << n << " x " << i << " = " << n * i << endl;
```

7 · LARGEST ELEMENT IN AN ARRAY

```
int arr[5] = {23, 78, 12, 90, 45};
int largest = arr[0];              // assume the first

for (int i = 1; i < 5; i++)
    if (arr[i] > largest) largest = arr[i];

cout << "Largest: " << largest << endl;   // 90
```

8 · LINEAR SEARCH

```
int arr[5] = {23, 78, 12, 90, 45}, target, pos = -1;
cin >> target;

for (int i = 0; i < 5; i++)
    if (arr[i] == target) { pos = i; break; }

if (pos != -1) cout << "Found at index " << pos << endl;
else           cout << "Not found" << endl;
```

9 · BUBBLE SORT

```
int a[5] = {5, 2, 9, 1, 7}, n = 5;

for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - 1 - i; j++)
        if (a[j] > a[j + 1]) {
            int t = a[j];  a[j] = a[j + 1];  a[j + 1] = t;
        }

for (int i = 0; i < n; i++) cout << a[i] << " ";  // 1 2 5 7 9
```

10 · COUNT VOWELS IN A STRING

```
string s;  int count = 0;
getline(cin, s);

for (int i = 0; i < s.length(); i++) {
    char c = tolower(s[i]);
    if (c=='a'||c=='e'||c=='i'||c=='o'||c=='u') count++;
}
cout << "Vowels: " << count << endl;
```

11 · STAR PATTERNS

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++)
        cout << "*";
    cout << endl;
}
```

```
*
**
***
****
*****
```

12 · SIMPLE CALCULATOR WITH `SWITCH`

```
double a, b;  char op;
cin >> a >> op >> b;

switch (op) {
    case '+': cout << a + b; break;
    case '-': cout << a - b; break;
    case '*': cout << a * b; break;
    case '/':
        if (b != 0) cout << a / b;
        else cout << "Division by zero";
        break;
    default: cout << "Invalid operator";
}
```

THE THREE PATTERNS BEHIND ALL TWELVE

1. The accumulator — declare a total at 0 (or a product at 1) **before** the loop, update it inside, print it after. Programs 2, 3, 5, 10.
2. The champion — assume the first element is the answer, then loop through the rest replacing it whenever you find better. Programs 1, 7.
3. The flag — declare a `bool`, set it to its default, flip it if the exceptional case is found, then decide after the loop. Programs 4, 8, and the login system in Volume II.
Recognise which of the three a question wants and the code writes itself.